BASH Programming

Get the absolute path of a script in Bash

1 year ago • by Kalyani Rajalingham

A path is a location to a directory or a file. There are two distinct types of paths in Linux – absolute and relative. The relative path is determined using the current working directory. On the other hand, the absolute path is the full path to a file or directory. The full path, in particular, is specified from the root directory. An easy way to remember which is by using the /. A relative path doesn't start with a / (the root folder). In this tutorial, we will learn how to get the absolute path of a script in Bash.

Absolute Path

To begin with, let's create a simple directory, sub-directories, and files.

```
kalyani@bournehacker:~/Desktop$ tree LinuxHint/
LinuxHint/
├── Project1
│   ├── PATH
│   │   └── path.txt
│   ├── PYTHON
│   │   └── python.txt
│   └── SCP
│       └── scp.txt
└── Project2
    └── script.sh

5 directories, 4 files
kalyani@bournehacker:~/Desktop$
```

In this example, if LinuxHint is our current working directory, then the absolute path of the

RELATED LINUX HINT POSTS

Copying Files and Directories in Linux

Is there a TRY CATCH command in Bash?

Read the CSV File in Bash

Create the Progress Bar in Bash

Bash Subshells

Bash Parallel Jobs Using For Loop

How to Resolve Bash Terminal Error: "Bash: Syntax Error Near Unexpected Token 'Newline'

file script.sh is:

```
/home/kalyani/Desktop/LinuxHint/Project2/script.sh
```

Our relative path is:

```
Project2/script.sh
```

What you can notice here is that in order to retrieve the file called script.sh, if we have an absolute path, we can retrieve it from anywhere in the Linux ecosystem. Our relative path is not as flexible; it, on the other hand, depends on the current working directory. In the previous case, if we were in the LinuxHint directory, and it was our current working directory, then to access the script.sh, we'd have to type in Project2/script.sh. Notice how there's no / at the beginning of the relative path.

Our goal is to retrieve the script's full address or path (absolute path).sh given a relative path.

ReadLink

One command that you can use to capture the full address of a file or an executable is readlink. Readlink is typically used to capture the path of a symbolic link or a canonical file. However, readlink can also compute the absolute path given a relative path. In all cases, you will need to attach a flag to readlink. The most commonly used flag in such cases is the f flag.

Example #1 – readlink using the f flag

```
script.sh
#!/bin/bash

path='readlink -f "${BASH_SOURCE:-$0}"'

DIR_PATH='dirname $path'

echo 'The absolute path is' $path
echo '----------------------------------------------'
echo 'The Directory Path is' $DIR_PATH
```

```
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$ cat script.sh
#! /bin/bash
```

```
path=`readlink -f "${BASH_SOURCE:-$0}"`

DIR_PATH=`dirname $path`


echo 'The absolute path is' $path
echo '-------------------'
echo 'The directory path is' $DIR_PATH
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$ ./script.sh
The absolute path is /home/kalyani/Desktop/LinuxHint/Project2/script.sh
-------------------
The directory path is /home/kalyani/Desktop/LinuxHint/Project2
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$
```

Typically, $0 is used to get the script's path; however, this doesn't always work. So a more reliable or robust way of getting the relative path of the script is by using ${BASH_SOURCE:-$0}.

Suppose for one instance that I write echo ${BASH_SOURCE:-$0}, the result I get is ./script.sh. This is the non-absolute path to our current script file. That is to say, the location of the script being executed is stored in ${BASH_SOURCE:-$0}.

```
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$ cat script.sh
#! /bin/bash

echo ${BASH_SOURCE:-$0}
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$ ./script.sh
./script.sh
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$
```

Once we reliably fetch the script's path, we can then pass it to readlink with the f flag. We can subsequently use the dirname command to fetch the directory path. The dirname command will omit the last value of the path and return the rest.

So if we have a path of /home/kalyani/Desktop/LinuxHint/Project2/script.sh, and we apply dirname to it, we will get /home/kalyani/Desktop/LinuxHint/Project2. This did strip the basename or the script's name from the address or path.

Realpath

Another command that can be used is realpath. Realpath is a Linux command used to print the resolved absolute file name. It requires that all components exist except for the last component.

```
script.sh
#!/bin/bash

path=$(realpath "${BASH_SOURCE:-$0}")
echo 'The absolute path is' $path

echo '-----------------------------------'

DIR_PATH=$(dirname $path)
echo 'The directory path is' $DPATH
```

Here, once again, we get the path of the script using ${BASH_SOURCE:-$0}. Realpath will fetch the full path for you, and dirname will get all but the last value of the absolute path.

Alternative #1

Now suppose that you didn't have the privilege of using realpath or readlink. It doesn't come with all Linux systems! I was lucky enough to have been using Ubuntu and thus could access it. However, a long way of doing the same thing is as follows:

```
script.sh
#!/bin/bash

DIR_PATH=$(cd $(dirname "${BASH_SOURCE:-$0}") && pwd)

path=$DIR_PATH/$(basename "${BASH_SOURCE:-$0}")

echo 'The absolute path is' $path
echo '-----------------------------------'
echo 'The directory path is' $DIR_PATH
```

```
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$ cat script.sh
#! /bin/bash

DIR_PATH=$(cd $(dirname "${BASH_SOURCE:-$0}") && pwd)

path=$DIR_PATH/$(basename "${BASH_SOURCE:-$0}")

echo 'The absolute path is' $path
echo '-------------------'
echo 'The directory path is' $DIR_PATH
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$ ./script.sh
The absolute path is /home/kalyani/Desktop/LinuxHint/Project2/script.sh
-------------------
The directory path is /home/kalyani/Desktop/LinuxHint/Project2
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$
```

In this case, first, we need the current script's path, and from it, we use dirname to get the directory path of the script file. Once we have that, we cd into the folder and print the working directory. To get the full or absolute path, we attach the basename of the script file to the directory path or $DIR_PATH.

Retrieving the path of another script (other than self)

In the previous examples, we retrieved the absolute and directory paths of the script file itself. What if we wanted to retrieve the absolute and directory paths of a file other than the script we're working in (other than self)?

```
kalyani@bournehacker:~/Desktop$ tree LinuxHint/
LinuxHint/
├── Project1
│   ├── PATH
│   │   └── path.txt
│   ├── PYTHON
│   │   └── python.txt
│   └── SCP
│       └── scp.txt
└── Project2
    ├── script2.sh
    └── script.sh

5 directories, 5 files
kalyani@bournehacker:~/Desktop$
```

So here, we've created a new file called script2.sh, and we'd like to get the absolute and directory paths of script2.sh.

In script.sh:

```
script.sh
#!/bin/bash

path=$(realpath script2.sh)
echo 'The absolute path is' $path

echo '----------------------------------'

DIR_PATH=$(dirname $path)
echo 'The directory path is' $DPATH
```

```
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$ cat script.sh
#! /bin/bash

path=$(realpath script2.sh)
echo 'The absolute path is' $path

echo '---------------------------'

DIR_PATH=$(dirname $path)
echo 'The directory path is' $DIR_PATH
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$ ./script.sh
The absolute path is /home/kalyani/Desktop/LinuxHint/Project2/script2.sh
---------------------------
The directory path is /home/kalyani/Desktop/LinuxHint/Project2
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$
```

Any of the previous methods should work here as well. However, here, we'll be using the relative path of script2.sh in order to retrieve the absolute path.

Retrieving the path of a command

Now, you can get the absolute and the directory paths of any scripts you want and that of commands. Let's assume for a moment that we want to get the absolute and directory paths of the command ls. We would write:

```
script.sh
#!/bin/bash

path=$(which ls)
echo 'The absolute path is' $path

echo '----------------------------------'

DIR_PATH=$(dirname $path)
echo 'The directory path is' $DIR_PATH
```

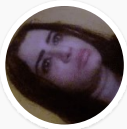
```
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$ cat script.sh
#! /bin/bash

path=$(which ls)
echo 'The absolute path is' $path

echo '---------------------------'

DIR_PATH=$(dirname $path)
echo 'The directory path is' $DIR_PATH
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$ ./script.sh
The absolute path is /usr/bin/ls
---------------------------
The directory path is /usr/bin
kalyani@bournehacker:~/Desktop/LinuxHint/Project2$
```

A path is an address used to locate a file or a folder. An absolute path is a full address or location such that no matter where you are, you can retrieve the file you want. On the other hand, a relative path is determined in relation to the current working directory. In bash, there are a number of ways of retrieving the full address of a script. In particular, we can use realpath, readlink, or even create our custom little script. When we want to know the directory path, we can use the dirname command in our bash script to retrieve our directory path. It is quite facile to obtain the full address using a relative address.

Happy Coding!

Explore More

ABOUT THE AUTHOR



Kalyani Rajalingham

I'm a linux and code lover.

View all posts